

Symbolic Verification of Detailed Balance in Monte Carlo Algorithms

S. Whitby

Abstract

We present *DetailedBalanceChecker*, a computer-algebra framework for the exact, symbolic verification of detailed balance in Monte Carlo (MC) algorithms. Every call to the random-number generator is intercepted and mapped to reads from a binary tape, enabling a breadth-first search (BFS) over all execution paths of the algorithm. The resulting symbolic transition matrix is tested against the detailed-balance condition algebraically, treating coupling constants, field functions, and inverse temperature as free symbols. A PASS constitutes a mathematical proof valid for all parameter values simultaneously; a FAIL identifies the exact symbolic expressions that violate balance for each offending state pair. The primary verification strategy exploits the algebraic structure of Metropolis-type acceptance: after expanding over sign cases of the coupling constants, each detailed-balance expression reduces to a sum of terms with common exponential factors, verified in microseconds by polynomial coefficient comparison. A canonical-ordering oracle and generator-based symmetry check enable exploitation of the lattice symmetry group G to compress the BFS and the verification, yielding near- $|G|$ -fold reductions in cost. Ergodicity is checked independently via BFS state-count comparison. Physical fidelity metrics quantify how closely the MC transition matrix matches reference Glauber dynamics. We demonstrate the framework on 1D Kawasaki dynamics, a rightward-only cluster algorithm that passes every numerical statistical test while violating detailed balance, and 2D Virtual Move Monte Carlo (VMMC) in both correct and broken variants. Intended use cases include regression testing, algorithm development, and as a formal oracle in an LLM-driven algorithm discovery loop. The implementation is available at <https://github.com/Sam-Whitby/CheckDetailedBalance2>.

1 Introduction

Monte Carlo (MC) simulation is the primary tool for sampling equilibrium distributions in statistical physics and soft matter [1, 3, 4]. The correctness of a MC algorithm rests on two conditions: ergodicity (every state is reachable from every other) and the *detailed balance* (DB) condition

$$T(i \rightarrow j) \pi(i) = T(j \rightarrow i) \pi(j), \quad (1)$$

where $T(i \rightarrow j)$ is the transition rate from microstate i to j and $\pi(i) \propto e^{-\beta E(i)}$ is the target Boltzmann weight. Together

with ergodicity, Eq. (1) guarantees convergence to the correct equilibrium distribution [6].

Algorithm correctness is overwhelmingly assessed by numerical tests. The most common approach runs a long Markov chain and compares the empirical stationary distribution to the Boltzmann distribution via the Kullback–Leibler (KL) divergence [5] or a χ^2 goodness-of-fit statistic. The joint-distribution test of Geweke [13] is the most principled numerical approach, comparing forward and time-reversed marginals of successive chain states. These tests share a fundamental limitation: they are statistical and therefore cannot certify correctness. They can only fail to detect a violation. An algorithm whose stationary distribution happens to be Boltzmann-distributed under the specific energy function and parameter values used in the test will pass every numerical test, even if it violates detailed balance and would sample incorrectly for a different energy — see Section 5.2 for a concrete example.

Analytical proofs of detailed balance for specific algorithms—Metropolis–Hastings [2], VMMC [8, 9], Swendsen–Wang [10], Wolff [11]—are algorithm-specific and laborious. They are also susceptible to the same sign errors and missed edge cases that numerical tests at benign parameters cannot detect. For complex cluster algorithms, where the acceptance criterion emerges from a multi-step cluster-growth procedure, the analytical proof requires careful book-keeping of all path weights, and subtle implementation errors may not be caught until the algorithm is applied to a model where the violation is not hidden by energy symmetries.

A more general reversibility condition—the Kolmogorov cycle criterion [14]—states that the chain is reversible if and only if the product of transition rates is equal in both directions around every closed loop of states. Checking this without knowing the stationary distribution requires either enumerating all cycles or matrix diagonalisation at $\mathcal{O}(N^3)$ cost [15], which becomes algebraically intractable for symbolic parameter values. When the energy E is known, as it always is in physics simulations, detailed balance reduces instead to $N(N-1)$ pairwise checks of Eq. (1)—a tractable algebraic problem.

We present an automated framework for the *exact symbolic* verification of Eq. (1). The framework intercepts every random call in the algorithm, constructs an exact symbolic transition matrix by BFS over all execution paths, and verifies detailed balance algebraically. All coupling constants and field functions are treated as free real symbols, so a PASS constitutes a mathematical proof valid for all param-

ter values simultaneously, including parameter combinations not accessible to numerical testing. A FAIL yields exact symbolic expressions identifying which transitions violate detailed balance and by how much, enabling diagnosis and correction. We also describe how symmetry of the state space under a lattice symmetry group G can be exploited to achieve near- $|G|$ -fold speedups in both the BFS and the verification step, extending the practical reach of the method.

2 Related Work

The dominant paradigm for MC correctness checking remains numerical. Histogram comparison against the Boltzmann distribution via KL divergence or χ^2 statistics [5] is standard practice, and the joint-distribution test of Geweke [13]—which compares forward and time-reversed marginals of successive chain states—is the most principled purely numerical approach. Both are statistical.

Formal methods for Markov chains have advanced considerably through *parametric model checking* [21], implemented in PRISM [22] and Storm [23]. In principle, steady-state probabilities computed by these tools could be combined with transition-matrix entries to check detailed balance. However, PRISM and Storm operate on models specified in dedicated input languages. Translating a Monte Carlo algorithm into such a format requires essentially the same case analysis as writing an analytic proof, and must be redone for each algorithmic variant. Detailed balance is not a native property of any standard temporal logic, so any verification requires additional post-processing outside the model-checker.

Exact symbolic inference for probabilistic programs—PSI [24] and similar tools—can in principle yield a single transition probability $T(i \rightarrow j)$ for a specific state pair. However, these systems target Bayesian inference, not exhaustive relational checking across all state pairs.

It is worth noting that detailed balance is *sufficient* but not *necessary* for a Boltzmann stationary distribution. The weaker global balance condition suffices [16], and algorithms such as event-chain Monte Carlo (ECMC) [17]—which achieves rejection-free updates with correct equilibrium statistics by satisfying only global balance via lifting variables—would *correctly* fail the detailed-balance checker. This is a feature: it defines a precise and demanding search objective.

To the authors’ knowledge, no existing framework provides automated detailed-balance checking as a first-class operation for Monte Carlo algorithms written as executable code, nor verifies correctness across the full parameter space in a single algebraic pass.

3 Theoretical Framework

We describe the algorithm in abstract terms, independent of any particular implementation.

3.1 Problem formulation

We consider a finite set of microstates $\mathcal{S}$ with a user-supplied energy function $E : \mathcal{S} \rightarrow \mathbb{R}$. An MC algorithm is a function $\mathcal{A} : \mathcal{S} \times \{0, 1\}^* \rightarrow \mathcal{S}$ mapping an initial state and a binary tape to a final state, where each random call inside $\mathcal{A}$ reads one or more bits from the tape. The transition probability from state i to state j is

$$T(i \rightarrow j) = \sum_{\mathbf{b}: \mathcal{A}(i, \mathbf{b})=j} w(\mathbf{b}), \quad (2)$$

where $w(\mathbf{b})$ is the probability of the bit sequence $\mathbf{b}$. The goal is to verify Eq. (1) symbolically, treating all model parameters (coupling constants, field values, inverse temperature β) as free real symbols.

3.2 State enumeration via BitsToState

The framework requires a user-supplied injective map $\text{BITSTOSTATE} : \{0, 1\}^* \rightarrow \mathcal{S} \cup \{\emptyset\}$ from binary strings to states, returning $\emptyset$ for strings that do not correspond to valid states. This induces a total ordering of states by bit-string length: shorter strings enumerate simpler states first. The checker iterates over all binary strings up to a user-specified maximum length, calling BITSTOSTATE on each, and launches a BFS from each valid seed state. Any newly discovered state becomes an additional seed.

For 2D square-lattice systems, the reference implementation provides a bijective encoding of labeled-particle configurations into integers: a non-negative integer ID encodes (i) the lattice size L^2 , via a prefix count over all arrays of smaller size; (ii) the particle number N , via a secondary prefix; (iii) the sorted particle positions, ranked by the combinatorial number system; and (iv) the assignment of particle types to positions, ranked by the factorial number system. Each step has an explicit inverse, so any integer can be decoded to the corresponding state. BITSTOSTATE interprets a bit string as the binary representation of ID and calls the decoder. Additional validity filters (e.g., requiring L^2 to be a perfect square for 2D grids) return $\emptyset$ for invalid IDs, causing the checker to skip those strings at zero BFS cost.

3.3 Reducing randomness to a binary tape

Every random call inside the algorithm is intercepted and replaced by a deterministic token that reads from a global binary tape $\mathbf{b} = b_1 b_2 \dots \in \{0, 1\}^*$. Four interception rules cover all supported call types.

Discrete uniform integers. Sampling from $\{l, \dots, h\}$ uses rejection sampling over blocks of $c = \lceil \log_2(h - l + 1) \rceil$ bits. A block B maps to outcome $l + B$ if $B < h - l + 1$; otherwise a new block is read. Summing over all retry paths gives each outcome weight $1/(h - l + 1)$ exactly, matching the target uniform distribution. This handles $\text{RandomInteger}[\{lo, hi\}]$ and uniform RandomChoice over a list.

Continuous uniform reals (interval tracking). A call $U \sim \text{Uniform}[0, 1]$ creates a symbolic token representing a latent variable U on the interval $[0, 1]$. Each subsequent comparison $U < p$ reads one bit: bit 0 restricts U to $[0, p]$ with probability weight $p/(h-l)$ (where $[l, h]$ is the current interval); bit 1 restricts to $[p, h]$ with weight $(h-p)/(h-l)$. Successive comparisons on the same variable—possibly to different thresholds in different branches—are all conditioned correctly on all prior constraints, implementing exact inverse-CDF sampling. This handles `RandomReal[]` and is critical for Metropolis acceptance steps, where a single U is compared to a single threshold but that threshold may itself be symbolic.

Weighted discrete choice. Sampling from a weighted distribution $\{(w_k, x_k)\}_{k=1}^n$ with $\sum_k w_k = W$ is decomposed into a sequential chain of Bernoulli trials using the binary representation of the cumulative distribution. Specifically, starting from the full weight sum W , the first trial asks whether $U < w_1/W$: bit 0 selects x_1 (with weight w_1/W); bit 1 continues to a trial on the remaining distribution. Each fresh trial uses an independent interval-tracking token. The weight of any complete path to outcome x_k is exactly w_k/W . This handles `RandomChoice[weights → elements]`.

Discretised Gaussian jumps. For algorithms on lattices where displacements are necessarily integer-valued, a normally distributed jump of standard deviation σ can be represented exactly. The probability that a continuous $\mathcal{N}(0, \sigma^2)$ random variable falls in the unit bin centred on integer d is

$$w(d) = \frac{1}{2} \left[\text{erf} \left(\frac{s_d + \frac{1}{2}}{\sqrt{2}\sigma} \right) - \text{erf} \left(\frac{s_d - \frac{1}{2}}{\sqrt{2}\sigma} \right) \right], \quad (3)$$

where s_d is the signed displacement on the ring. The weights are symmetric ($w(d) = w(-d)$), normalised, and sum to one over all integer displacements within a truncation range. The displacement is drawn via weighted discrete choice with these weights, which the checker decomposes into sequential Bernoulli trials. The resulting symbolic transition matrix entries contain `erf` functions with σ as a free parameter, and detailed balance is verified for all values of σ simultaneously.

All four reductions are exact: integrating the branch weights over all bit sequences recovers the original probability measure of each random call. Calls of each type may be combined arbitrarily within a single algorithm execution; the symbolic weight of any complete path is the product of the weights of all individual bits read.

3.4 BFS construction of the transition matrix

Algorithm 1 constructs the symbolic transition matrix by breadth-first search. Starting from a seed state s_0 , the algorithm $\mathcal{A}$ is re-executed for each distinct binary prefix in the queue. When a random call is encountered, execution branches into two continuations (one per bit value), each pushed with its respective probability weight. When execution completes, the output state s' and accumulated weight w contribute $T[s_0 \rightarrow s'] += w$. Each newly discovered

Algorithm 1 Symbolic transition-matrix construction (BFS)

Require: Algorithm $\mathcal{A}$, seed state s_0 , energy E

```

1: queue  $\leftarrow \{(s_0, \varepsilon, 1)\}$ ;  $T \leftarrow \mathbf{0}$ ; seen  $\leftarrow \{s_0\}$ 
2: while queue not empty do
3:    $(s, \mathbf{b}, w) \leftarrow \text{queue.pop}()$ 
4:   Run  $\mathcal{A}(s)$  with tape  $\mathbf{b}$ , intercepting random calls
5:   if execution completes with output  $s'$  then
6:      $T[s \rightarrow s'] += w$ 
7:     if  $s' \notin \text{seen}$  then
8:       Add  $s'$  to seen; seed new BFS from  $s'$ 
9:     end if
10:  else (next bit  $b_i$  needed)
11:    Push  $(s, \mathbf{b} \cdot 0, w_0)$  and  $(s, \mathbf{b} \cdot 1, w_1)$ 
12:  end if
13: end while
14: return  $T$ 
```

state becomes an additional seed. The procedure simultaneously constructs the transition matrix and discovers the full connected component reachable from any seed.

The BFS tree for a single state has depth equal to the maximum number of random bits consumed by any complete execution path. For a Metropolis algorithm choosing among n discrete options with Gaussian-displacement acceptance, this is typically $\lceil \log_2 n \rceil + O(1)$ bits per cluster step, giving a tree of size $\mathcal{O}(2^{\lceil \log_2 n \rceil})$ per state. The total cost grows as $\mathcal{O}(N^2 \cdot 2^d)$ where N is the component size and d is the maximum bit depth, so the method is practical only for small systems (typically $N \lesssim 1000$, $d \lesssim 20$).

3.5 Detailed balance verification

Given the symbolic transition matrix T and energy function E , the checker evaluates

$$\Delta_{ij} = T(i \rightarrow j) e^{-\beta E(i)} - T(j \rightarrow i) e^{-\beta E(j)} \stackrel{?}{=} 0 \quad (4)$$

for every ordered pair (i, j) . Here β , all coupling functions $J(\cdot, \cdot, \cdot)$, field functions $f(\cdot, \cdot)$, and any other model parameters are free real symbols. Each distinct call $J(a, b, d^2)$ or $f(x, y, L)$ is treated as an independent free real atom, so a PASS proves correctness for *all* possible energy models, not merely a specific parametric form.

Three efficiency steps are applied before any algebra.

Trivial-zero filter. Pairs where both $T(i \rightarrow j)$ and $T(j \rightarrow i)$ are structurally zero (never connected by the algorithm) are skipped without any algebraic work.

Syntactic deduplication. All remaining Δ_{ij} expressions are hashed before simplification. Each unique expression is simplified exactly once; structurally identical copies share the result. For periodic lattices with translational symmetry, this alone reduces thousands of unique pairs to hundreds—e.g., approximately $9\times$ for a 3×3 torus. With the additional canonical-ordering oracle (Section 3.6), G-orbit pairs produce syntactically identical expressions, enabling a further $\sim |G|$ -fold collapse.

FastChecker: polynomial coefficient test. The primary simplification strategy exploits the algebraic structure of Metropolis acceptance. After expanding a Δ_{ij} expression over sign cases of the coupling constants via **PiecewiseExpand**, each case reduces (within each feasible sign regime) to a sum of terms $c_k e^{-\beta L_k}$, where c_k is a rational coefficient and L_k is a polynomial in the coupling constants. Detailed balance holds if and only if, for each distinct exponent polynomial L_k , the sum of its coefficients vanishes:

$$\sum_{k: L_k=L} c_k = 0 \quad \forall L. \quad (5)$$

This is verified by pure polynomial arithmetic in microseconds.

Before applying the test, each **Piecewise** case must be checked for *feasibility*—whether there exist real values of the coupling constants satisfying the case conditions. The infeasible cases cannot contribute violations. The feasibility check uses a two-path strategy: (1) a fast $\mathcal{O}(n)$ pattern-match identifies conditions consisting purely of **Unequal** atoms between distinct expressions, which are always satisfiable over the reals (each constraint is a co-dimension-1 surface; any finite conjunction of such surfaces has full Lebesgue measure); (2) for conditions not covered by the fast path, **FindInstance** locates a concrete example, which is typically 10–100 $\times$ faster than **Reduce** for polynomial-inequality conditions.

Implicit equality extraction (deepCheck). For the rare case where the fast path is inconclusive after simplification, a second pass (**deepCheck=True**) is applied. **LogicalExpand** first expands any negated conjunctions in the **Piecewise** conditions (e.g., $\neg(A \wedge B) \rightarrow \neg A \vee \neg B$), exposing implicit equalities hidden inside negated conditions. **SubstEqualities** then substitutes these equalities into the residual expression; algebraic cancellation then confirms zero without further calls to **FullSimplify**. This step was required to avoid a false positive in the VMMC algorithm at 3×3 grid size: without it, **FullSimplify** treated each **Piecewise** case independently and missed the implicit constraint $J_{231} = J_{232}$ implied by prior-case exclusion, erroneously reporting 1296 spurious violations.

FullSimplify fallback. Any expression for which the polynomial test and **deepCheck** are both inconclusive falls back to **FullSimplify**[**PiecewiseExpand**[\(\Delta_{ij}\), \(\{\beta > 0, \dots\}\)] with sign assumptions on coupling constants. This is always exact, at the cost of higher computational time (typically 0.3–5s per expression).

3.6 Symmetry-aware checking

For algorithms on periodic lattices whose energy and proposal are invariant under a lattice symmetry group G , the detailed-balance expressions Δ_{ij} and $\Delta_{i'j'}$ are equal whenever (i', j') is a G -orbit image of (i, j) . Checking one representative per orbit would suffice, giving an $|G|$ -fold speedup. For the square torus with full D4 symmetry (8 rotations and reflections) combined with $(L \times L)$ translations, $|G| = 8L^2$,

making the speedup significant for any lattice of practical size.

The challenge is that G -orbit pairs reference different site indices and different neighbour-list orderings, producing syntactically distinct symbolic expressions even when mathematically equal. The existing syntactic deduplication therefore does not automatically group them. We solve this by making the BFS itself produce *syntactically identical* expressions for G -orbit pairs.

Canonical-ordering oracle. The seqBernoulli decomposition tree produced by the cluster-building routine is syntactically determined by the *order* in which candidate neighbour sites are visited. For G -invariant algorithms, the topological key

$$(d^2(p, q), d^2(p_{\text{post}}, q), d^2(p_{\text{rev}}, q), \text{type}(q))$$

where d^2 denotes the minimum-image squared distance, is preserved by every G -isometry. Sorting candidates by this key assigns the same canonical order to all G -orbit-related states. The BFS then produces the same symbolic tree for every state in a G -orbit, making the resulting Δ_{ij} expressions syntactically identical and hash-equivalent. The existing syntactic deduplication then collapses the entire orbit to one representative evaluated once, at no additional verification cost.

The canonical oracle is exposed through the standard interface function **\$vmcCandidates** defined in the shared geometry layer. The checker substitutes the canonical implementation via Mathematica’s **Block** during BFS: the algorithm requires no modification and remains unaware of the substitution:

```
$bfsAlg = Function[s,
  Block[{$vmcCandidates =
    $dbcCanonicalCandidates},
    $alg[s]]]
```

G-invariance verification. After BFS, a separate check verifies that the computed T matrix is indeed G -invariant: $T(s \rightarrow s') = T(g(s) \rightarrow g(s'))$ for all generators g and all state pairs. The group $G = (\mathbb{Z}/L\mathbb{Z})^2 \rtimes D_4$ on an $L \times L$ torus is generated by four elements: **translate**(1, 0), **translate**(0, 1), **rotate**₉₀, and **reflect** (main diagonal). G -invariance under any product of generators follows by induction if it holds for all four; thus four sweeps over all state pairs suffice to certify invariance under all $|G| \leq 8L^2$ elements. Each generator’s action on all states is precomputed once into a hash table, making per-pair lookups $\mathcal{O}(1)$. The comparison is syntactic equality (valid because the canonical ordering makes G -related entries syntactically identical).

If the G -invariance check fails—indicating the declared symmetry group is incorrect for this algorithm—the checker reports **FAIL** and the DB result is unaffected (it is always correct regardless of the symmetry declaration). If it passes, the user has both a formal proof of DB and a certificate of G -invariance.

Table 1 summarises the safety guarantees.

Table 1: Safety guarantees for symmetry-aware checking. The DB check result is always correct. G-invariance failure warns that the declared symmetry is wrong; it does not produce false DB results.

Scenario	DB	G-inv
Satisfies DB and G	PASS	PASS
Violates DB	FAIL	PASS/FAIL
Satisfies DB but not G	PASS	FAIL
Wrong \$symmetryGroup declaration	PASS	FAIL

Conditions for a valid symmetry declaration. Three structural conditions must hold for the G -invariance declaration to be correct and the orbit reduction to give a valid speedup: (1) seed selection is uniform over occupied sites with no position-dependent weighting; (2) the displacement list contains every (dx, dy) paired with $(-dx, -dy)$ (proposal symmetry $P(\mathbf{d}) = P(-\mathbf{d})$); (3) the energy uses only minimum-image periodic distances (no external field). If any condition fails, the G -invariance check will detect and report it.

3.7 Ergodicity checking

Detailed balance alone does not guarantee a correct sampler; the chain must also be ergodic (every state reachable from every other state). The checker verifies ergodicity independently of detailed balance by comparing the number of states discovered in each BFS-connected component with the theoretical state count. For N labeled particles on S sites (with at most one particle per site), the number of reachable configurations from any starting configuration under a swap-type move is the falling factorial $(S)_N = S(S-1)\cdots(S-N+1)$. A component whose BFS-discovered state count equals $(S)_N$ is ergodic; a smaller count signals non-ergodicity.

This check is implemented by the `ValidStateIDs` function in each algorithm file, which specifies the theoretical count for the given system size, and by comparing BFS-discovered state counts to this value. A separate ergodicity check based on the support matrix $(I + S)^{n-1}$ —which has a strictly positive (i, j) entry if and only if state j is reachable from i [20]—is available for small components where matrix-power computation is feasible.

Notably, detailed balance does not imply ergodicity and vice versa. A non-ergodic algorithm can satisfy DB within each disconnected component while failing to reach all of state space; this is demonstrated explicitly in Section 5.

3.8 Physical fidelity metrics

Beyond the binary DB and ergodicity checks, the checker computes two optional scalar metrics measuring how closely the algorithm’s transition matrix T_{MC} matches a physical reference dynamics. For a 2D periodic square lattice, the

physical reference is Glauber single-particle dynamics [18]. For each state pair (i, j) differing by exactly one nearest-neighbour hop of a single particle:

$$T_{\text{phys}}(i \rightarrow j) = \frac{1}{N} \cdot \frac{1}{z} \cdot \frac{1}{1 + e^{\beta \Delta E}}, \quad (6)$$

where $z = 4$ is the 2D coordination number and $\Delta E = E(j) - E(i)$. All other entries of T_{phys} are zero (diagonal entries enforce row-sum normalisation). T_{phys} is constructed exactly from the enumerated state space without simulation.

M1: relaxation spectrum ratio.

$$M_1 = \left| \frac{\lambda_2^{\text{MC}}}{\lambda_3^{\text{MC}}} - \frac{\lambda_2^{\text{phys}}}{\lambda_3^{\text{phys}}} \right| \quad (7)$$

where eigenvalues are sorted by real part (descending), $\lambda_1 = 1$ for any ergodic chain. M_1 measures whether the algorithm reproduces the correct hierarchy of relaxation timescales. $M_1 = 0$ indicates the ratio of the two slowest relaxation rates matches Glauber exactly; larger M_1 indicates a distorted timescale structure.

M2: row-normalised KL divergence (fidelity score).

$$M_2 = - \sum_i \pi_i \sum_j T_{\text{phys}}(i \rightarrow j) \log \frac{T_{\text{phys}}(i \rightarrow j)}{T_{\text{MC}}(i \rightarrow j)}, \quad (8)$$

the π -weighted average KL divergence of the physical transition distribution from the MC distribution, with the sign convention that $M_2 = 0$ is a perfect match and $M_2 < 0$ indicates departure. A value of $-\infty$ (hard failure) indicates the algorithm cannot execute a transition that is physically required, $T_{\text{MC}}(i \rightarrow j) = 0$ for some (i, j) with $T_{\text{phys}}(i \rightarrow j) > 0$.

These metrics are not pass/fail criteria. A more negative M_2 for a cluster algorithm such as VMMC is expected and reflects algorithmic acceleration: the algorithm decorrelates faster than physical Brownian dynamics by construction. The metrics provide a quantitative basis for comparing the *kinetic character* of different MC moves targeting the same equilibrium distribution.

3.9 Limitations

Termination. There is no guarantee that the BFS terminates for an arbitrary algorithm. Algorithms generating unbounded random sequences correspond to infinite binary trees; no automated procedure can decide termination in general (the halting problem [19]). BFS branches are truncated at a user-specified maximum depth (default 20 bits); paths reaching this depth are silently excluded. A warning is printed whenever truncation occurs; false negatives (missed violations) are possible in this regime.

Energy function. The checker uses `energy[state]` exactly as written. An incorrect energy function invalidates both the symbolic and numerical checks simultaneously. Random calls inside the energy function are detected and flagged, but

Table 2: Known failure modes. False negatives are cases where a broken algorithm passes; false positives are cases where a correct algorithm fails. Most risks are mitigated by design choices or warnings.

Source	Description	Mitigation
<i>False negatives</i>		
MaxBitDepth	Paths > 20 bits excluded	Increase option; WARNING
TimeLimit	BFS exits after 120 s/state	WARNING printed
MaxComponents	Only first N components checked	Set appropriately
Hidden state	Mutable globals contaminate paths	Stateless design
<i>False positives</i>		
FullSimplify incomplete	Rare; catches most	deepCheck Two-pass strategy
Unlisted params	Numeric value reduces check to single point	List all in <code>\$checkerAbstractParams</code>

other incorrectness (wrong formula, wrong sign) is invisible to the framework.

Hidden state. If the algorithm uses mutable global state, system time, or external calls that are not intercepted, the recorded execution paths may not match actual execution. `CheckAlgorithmSafety` detects known unsafe call patterns but cannot detect custom C extensions or arbitrary I/O.

Table 2 summarises the known false-negative and false-positive risks.

4 Computational Implementation

4.1 Mathematica as the host language

The framework’s central requirement is that every call to the random-number generator can be silently intercepted and replaced by a symbolic token, without the algorithm under test being modified or even aware of the substitution. Mathematica satisfies this via `Block[{f=g,...}, expr]`, which dynamically rebinds any named function for the duration of a lexical scope. Regardless of how deeply a call to `RandomReal[]` is buried inside utility functions or conditional branches, `Block` intercepts it transparently; the algorithm requires no cooperation. No other mainstream language provides this mechanism for arbitrary runtime call trees without source modification.

The second essential feature is Mathematica’s *UpValue* system, which allows comparison operators (`<`, `≤`, ...) to be overloaded on the interval-tracking token *within the CAS evaluator itself*: the expression `RandomReal[] < MetropolisProb[dE]` dispatches to the symbolic token handler via `UpValue` without any special syntax in the algorithm. Together, `Block` and `UpValues` make the interception layer completely invisible to the algorithm author, eliminating the most dangerous class of

false negatives: those where a random call escapes symbolic tracking and returns a concrete floating-point number that superficially resembles a valid symbolic weight.

For the algebraic verification step, Mathematica’s `PiecewiseExpand` reduces products of `Piecewise` functions to a flat list of cases by computing the Cartesian product of all condition branches automatically. `Reduce[cond ∧ β > 0, params, Reals]` provides a complete decision procedure for linear real arithmetic over coupling constants. When these are exhausted, `FullSimplify` applies a search over hundreds of algebraic transformation rules, handling mixed exponential-`Piecewise` expressions that arise in complex algorithms such as VMMC with abstract coupling fields.

Alternative implementations. The strongest open-source alternative is Julia combined with Symbolics.jl [27]. The Cassette.jl overdudding framework [28] provides a runtime interception mechanism close to Mathematica’s `Block`: a context can transparently redirect calls to `rand()` anywhere in a Julia call tree to a user-defined symbolic substitute. A Julia implementation would remove the proprietary-software dependency while preserving most robustness advantages. Python with SymPy [25] is the weakest option: it lacks a runtime overdudding mechanism, SymPy’s `piecewise_fold` does not perform cross-product expansion, and `ask` is incomplete for multivariate linear inequalities.

4.2 Computational cost and parallelisation

The two main computational bottlenecks are the BFS itself and the `FullSimplify` calls.

Parallel BFS. `BuildTreeAT` uses level-synchronous parallel BFS: the entire current wave frontier is sent to `ParallelMap` in one call, partitioned into at most n_K chunks (one per available kernel). This gives at most one round-trip overhead per BFS wave, amortised across all states in the wave. Single-state waves revert to sequential evaluation automatically. On a 4-core machine, this yields a $3.3\times$ speedup for the 504-state `vmmc_continuous.wl` component at `NGrid=3`: sequential BFS time 96.3 s reduced to 28.9 s.

Parallel FullSimplify. When the fast check is inconclusive for many unique expressions, the fallback `FullSimplify` is also parallelised. All unique expressions requiring `FullSimplify` are collected after syntactic deduplication and distributed across available subkernels in one `ParallelMap` call. The subsequent violation scan performs only fast Association lookups. For `vmmc_continuous.wl` at `NGrid=3`: total check time reduced from 29 min to 12.5 min on 4 kernels.

Symmetry-aware cost savings. The canonical-ordering oracle and hash deduplication together achieve near- $|G|$ -fold compression. For a two-particle component on the 3×3 torus (72 states, $|G| = 72$ under D4+translations), the 288 non-trivial (i, j) pairs collapse to 2 unique expressions: 99% of all `FullSimplify` evaluations are eliminated. The G-invariance check over the full 504-state component takes 4.3 s on 4 kernels. For reference, Table 3 shows estimated

Table 3: Estimated runtimes for the `vmmc.lattice.wl` 504-state component at `NGrid=3` on 4 kernels, as a function of symmetry exploitation. FS = FullSimplify calls; estimates scale linearly from measured 12.5 min at no-symmetry.

Mode	Pairs	FS calls	Est. time
No symmetry	126,756	~300	~12 min
Translations ($ G = 9$)	14,084	~33	~1.4 min
D4+trans. ($ G = 72$)	~1,760	~4	~10 s

runtimes at `NGrid=3` under different symmetry exploitation levels (estimated from measured data).

4.3 User interface and algorithm files

Users supply a `.wl` algorithm file defining four items: the energy function `energy[state]`, the MC move `Algorithm[state]`, the BitsToState decoder `BitsToState[bits]`, and a numeric inverse temperature `numBeta` for the numerical check. Optional definitions control symmetry declarations, abstract parameter handling, and display formatting.

Abstract parameters. Physical parameters (e.g., `physLen`, `epsLJ`) that must be unbound symbols during BFS are declared as string names in `$checkerAbstractParams`. The checker saves their concrete values, clears the symbols, runs BFS, then restores values for the numerical MCMC check. String names are required because a list of symbol references would evaluate to their current numeric values before the checker intercepts them.

Abstract coupling functions. When a coupling function `couplingJ` should have no concrete definition during BFS (each call treated as a free real atom), a concrete implementation `$pairEnergy` is provided separately. The checker injects it via `Block[{couplingJ}, couplingJ := $pairEnergy]` for numerical runs.

The checker is invoked as:

```
wolframscript -file check.wls algo.wl \
  NGrid=N MaxComponents=N Mode=Symbolic
```

with options controlling grid size, component limits, bit depth, symmetry exploitation, and parallelisation. A report script (`report.wls`) provides detailed output for a single seed state, including the full decision tree and symbolic transition matrix.

5 Results

5.1 1D Kawasaki dynamics on a ring

The 1D Kawasaki algorithm [7] operates on a periodic ring of L sites. Each site holds a vacancy (type 0) or one of N_t particle types. At each step, two adjacent sites are chosen uniformly at random and their contents swapped with

Table 4: Representative symbolic transition matrix entries for the 4-site 1D Kawasaki ring ($L=4$, particles of types 1 and 2). States shown as periodic lattice rows $\begin{bmatrix} \cdot & \cdot & \cdot & \cdot \end{bmatrix}$. All non-zero off-diagonal entries follow the same pattern.

Transition ($i \rightarrow j$)	Description	$T(i \rightarrow j)$
<i>From adjacent pair ($E = J_{12}$)</i>		
$\begin{bmatrix} 0 & 0 & 1 & 2 \end{bmatrix} \rightarrow \begin{bmatrix} 0 & 0 & 2 & 1 \end{bmatrix}$	swap types; $\Delta E = 0$	$\frac{1}{4}$
$\begin{bmatrix} 0 & 0 & 1 & 2 \end{bmatrix} \rightarrow \begin{bmatrix} 0 & 1 & 0 & 2 \end{bmatrix}$	separate; $\Delta E = -J_{12}$	$\frac{1}{4} \min(1, e^{\beta J_{12}})$
$\begin{bmatrix} 0 & 0 & 1 & 2 \end{bmatrix} \rightarrow \begin{bmatrix} 2 & 0 & 1 & 0 \end{bmatrix}$	separate (wrap); $\Delta E = -J_{12}$	$\frac{1}{4} \min(1, e^{\beta J_{12}})$
<i>From separated pair ($E = 0$)</i>		
$\begin{bmatrix} 0 & 1 & 0 & 2 \end{bmatrix} \rightarrow \begin{bmatrix} 0 & 0 & 1 & 2 \end{bmatrix}$	bring adjacent; $\Delta E = +J_{12}$	$\frac{1}{4} \min(1, e^{-\beta J_{12}})$
$\begin{bmatrix} 0 & 1 & 0 & 2 \end{bmatrix} \rightarrow \begin{bmatrix} 0 & 1 & 0 & 2 \end{bmatrix}$	self (failed proposals)	$1 - \min(1, e^{-\beta J_{12}})$

Metropolis probability $\min(1, e^{-\beta \Delta E})$, where the energy is the nearest-neighbour sum

$$E(\mathbf{s}) = \sum_{i=1}^L J(s_i, s_{i \bmod L+1}, 1), \quad (9)$$

with $J(a, b, d^2)$ a coupling function treated as a free symbol for each distinct type pair.

For a 4-site ring with one particle of type 1 and one of type 2, the checker discovers 12 states, which fall into two energy classes: 8 states with the particles adjacent (energy J_{12}) and 4 with them separated (energy 0). Table 4 shows representative transition matrix entries. The detailed-balance condition for the pair $(\begin{bmatrix} 0 & 0 & 1 & 2 \end{bmatrix}, \begin{bmatrix} 0 & 1 & 0 & 2 \end{bmatrix})$ evaluates to

$$\frac{1}{4} \min(1, e^{\beta J_{12}}) \cdot e^{-\beta J_{12}} = \frac{1}{4} \min(1, e^{-\beta J_{12}}) \cdot 1, \quad (10)$$

which holds for all J_{12} : if $J_{12} > 0$, both sides equal $\frac{1}{4} e^{-\beta J_{12}}$; if $J_{12} < 0$, both equal $\frac{1}{4}$. All 132 non-trivial Δ_{ij} simplify to zero and the checker reports PASS.

5.2 Rightward-only cluster: a false negative in numerical testing

We demonstrate the diagnostic power of symbolic checking over numerical testing using a one-dimensional cluster algorithm that always violates detailed balance yet passes every statistical test.

The algorithm identifies all maximal connected clusters of particles on the ring. One cluster is chosen at random. If the site immediately clockwise of the cluster is vacant, the entire cluster is slid one step clockwise and accepted with Metropolis probability $\min(1, e^{-\beta \Delta E})$. No leftward slide is ever proposed.

For any transition $i \rightarrow j$ caused by a rightward slide, $T(j \rightarrow i) = 0$ because the reverse leftward move is never proposed. Thus $\Delta_{ij} = T(i \rightarrow j) e^{-\beta E(i)} \neq 0$ for all J_{ab} and β . The symbolic checker detects this immediately.

Crucially, however, the algorithm visits states in a directed cycle under rigid cluster translation. Because sliding a cluster rigidly preserves all internal bonds and simply relabels the boundary bond, every state in the cycle has the same energy. The Boltzmann distribution over the cycle is

Table 5: Checker output (Mode=Both) for the rightward-only 1D cluster algorithm. #St: component size. Sym: symbolic result. #F: violated ordered pairs. KL: Kullback–Leibler divergence from Boltzmann (100 000 steps). Dash: single-state component.

Bits	State	#St	Sym.	#F	KL	Num.
10	{1}	1	PASS	0	–	–
100	{1, 0}	2	PASS	0	~0	PASS
110	{1, 2}	1	PASS	0	–	–
1001	{1, 0, 0}	3	FAIL	3	~0	PASS
1100	{1, 2, 0}	3	FAIL	3	~0	PASS
11001	{1, 0, 0, 0}	4	FAIL	4	~0	PASS
11101	{1, 2, 0, 0}	4	FAIL	4	~0	PASS
101001	{1, 2, 3, 0}	4	FAIL	4	~0	PASS

therefore uniform, and any numerical test comparing the empirical histogram to the Boltzmann distribution finds perfect agreement.

Table 5 shows checker output (Mode=Both) for several seed states. States where all sites are occupied form single-state trivially balanced components. For all multi-state components, the symbolic checker finds violations while the numerical KL divergence is consistent with zero and the numerical check reports PASS.

This example illustrates a broad class of latent bugs: an algorithm whose stationary distribution is exactly Boltzmann under translation-invariant energies, yet which violates detailed balance and would sample incorrectly for any position-dependent energy (e.g., an external field). No numerical test at the symmetric energy can detect the failure; the symbolic check catches it unconditionally, for all parameter values, without running any Markov chain.

5.3 2D Virtual Move Monte Carlo

VMMC [8] is a cluster algorithm for particles on a 2D periodic lattice. A seed particle is chosen uniformly at random, a virtual rigid displacement in a randomly chosen direction is proposed, and a cluster is grown by recursively testing each occupied neighbour with forward and reverse link probabilities

$$p_{\text{fwd}}(i, j) = \max(0, 1 - e^{\beta(\varepsilon_{\text{init}} - \varepsilon_{\text{fwd}})}), \quad (11)$$

and analogously p_{rev} . A frustrated reverse link aborts the move; otherwise the cluster translates rigidly. This enforces detailed balance without a Metropolis acceptance step [8].

Correct VMMC. On a 2×2 torus with 2 distinct particles (12 states), the checker verifies all 132 non-trivial Δ_{ij} and reports PASS. A representative entry is

$$T(\{0, 0, 1, 2\} \rightarrow \{1, 2, 0, 0\}) = \frac{1}{2}(1 - (1 - p_{12})^2), \quad (12)$$

where $p_{12} = \max(0, 1 - e^{-\beta J_{12}})$, reflecting the two cluster orientations through which this transition can be assembled.

Missing-direction variant. An intentionally broken variant omits the “up” direction. On a 2×2 torus, periodic

boundary conditions render up and down equivalent, so the asymmetry is invisible and the checker correctly reports PASS. On a 3×3 torus, seeding with a two-particle state, the checker finds 144 violated pairs—the broken directional symmetry is unambiguously exposed.

Biased-displacement variant. `vmmc_biased.wl` duplicates rightward displacements so that $P(dx > 0, dy) = 2 \cdot P(dx < 0, dy)$. This breaks DB (correctly caught: FAIL) and also breaks G -invariance under D4 (the G -invariance check reports FAIL for the `rotate90` generator, since a 90° rotation maps the over-represented rightward moves to unbiased upward moves).

Abstract field and coupling. A further variant leaves both $J(a, b, d^2)$ and an external field function $f(x, y, L)$ completely unevaluated during the symbolic check. Every call is treated as an independent free real atom. A PASS proves correctness for all possible energy models simultaneously. On a 2×2 grid with 3 distinct particles (24 states), the checker reports PASS under these fully abstract conditions.

5.4 Symmetry-aware checking at NGrid=3

`vmmc_lattice.wl` replaces the rounded-Gaussian displacement of `vmmc_continuous.wl` with a uniform box $\{(dx, dy) : |dx| \leq n_{\text{step}}, |dy| \leq n_{\text{step}}\} \setminus \{(0, 0)\}$. This ensures $P(\mathbf{d}) = P(-\mathbf{d})$ structurally and makes all seqBernoulli weights rational constants, eliminating `erf` functions and making the FastChecker complete (no false-positive risk from transcendental feasibility conditions). The algorithm declares `$symmetryGroup = {"translation", "D4"}`.

At NGrid=3 (MaxComponents=4, 4 kernels):

- The two-particle component (72 states) produces 288 non-trivial (i, j) pairs, which collapse via hash deduplication to **2 unique expressions** (99% collapse rate).
- The G -invariance check over the 504-state component takes 4.3 s.
- Total wall time for all checked components: ≈ 2 min 55 s.

The checker reports DB: PASS, Ergodicity: PASS (state count matches $(S)_N$ for every component), G -invariance: PASS.

Ergodicity failure example. `kawasaki_1d_nonergodic.wl` demonstrates the independence of the two checks. Only particles of type 1 participate in swaps; particles of types 2 and higher are frozen. The checker reports: DB PASS (within each invariant subspace the swap dynamics are correctly balanced), Ergodicity FAIL (state count smaller than $(S)_N$ because frozen particles cannot rearrange). Detailed balance cannot detect non-ergodicity; both checks are required for a valid sampler.

6 Intended Use Cases

DetailedBalanceChecker is designed for algorithm developers and simulation practitioners in statistical physics, soft matter, and computational chemistry. Below we describe the principal intended use cases and their scope.

6.1 Verification and regression testing

The most immediate use is as a regression test for existing MC algorithms. A single invocation of `check.wls` over the smallest non-trivial system sizes (typically 2×2 or 3×3 for 2D algorithms) provides a formal proof of correctness for all parameter values simultaneously. This surpasses statistical testing for two reasons: (i) a PASS is a theorem, not a statistical outcome, and (ii) the proof covers the full parameter space, including parameter regimes—e.g., near a phase transition, or with unusual coupling ratios—where numerical tests would require a dedicated targeted run.

The tool catches bugs that are invisible to numerical tests. The rightward cluster example (Section 5.2) is one class; others include: asymmetric proposal distributions that happen to cancel at specific temperatures; missing reverse moves that only matter in the presence of an external field; energy function sign errors that are compensated by an incorrect acceptance rule.

As a regression tool, the checker can be integrated into a continuous integration pipeline: on any modification of the algorithm file, a fast symbolic check (`Mode=Symbolic`, `MaxBitString=11111`) runs in under two minutes and reports any newly introduced violations before code is merged.

6.2 Algorithm development: design, certify, translate

For new algorithm development, the natural workflow is to design and certify in the Mathematica environment before translating to a production language. The algorithm is implemented as a `.wl` file; the checker provides formal verification on small systems. Once a PASS is obtained, the algorithm is translated to C++, Python, Julia, or Fortran for production use.

This workflow addresses a known challenge in simulation algorithm design: detailed-balance proofs for complex cluster algorithms require careful tracking of many forward and reverse path weights, and are susceptible to subtle sign errors and missed edge cases. The checker replaces this manual book-keeping with an automated algebraic verification.

The `report.wls` script provides additional diagnostic support: for a single seed state it prints the full BFS decision tree, the symbolic transition matrix, and identifies all violated pairs. This allows the developer to understand exactly which paths contribute to each transition probability and why the balance condition fails, guiding targeted correction.

Translation of a certified algorithm to a production language introduces a gap: the checker proves the Mathematica implementation satisfies detailed balance, not the

C++/Python translation. Two strategies mitigate this. First, for simple algorithms (single-particle Metropolis, standard Kawasaki), the subset of Mathematica used—integer arithmetic, array indexing, conditional branches—maps mechanically to any language, and the translation can be validated by running both on identical random seeds. Second, for complex algorithms, the Mathematica implementation serves as a reference: a test suite comparing the two implementations on a comprehensive set of states and random seeds establishes the translation’s fidelity with high but not absolute confidence.

6.3 LLM-assisted algorithm discovery

A prospective use case is as the correctness oracle in a large language model (LLM) algorithm discovery loop [30, 31]. The proposed architecture is as follows: (1) an LLM generates a candidate MC algorithm as a Mathematica file; (2) the checker runs symbolically and returns a binary PASS/FAIL; (3) on FAIL, the exact symbolic expression $\Delta_{ij} = T(i \rightarrow j)\pi(i) - T(j \rightarrow i)\pi(j) \neq 0$ for the violating state pair is fed back verbatim to the LLM; (4) the LLM revises the algorithm and the loop repeats.

The symbolic counterexample provides qualitatively richer feedback than any runtime error message. A Python traceback says “your code crashed at line 12”; the checker says “the detailed-balance condition fails for states (3, 7) because the expression $e^{\beta J_{12}} - e^{\beta J_{13}}$ is nonzero”. This is a mathematical statement about the structure of the algorithm—identifying which part of the growth rule generated the J_{12} term and which generated the J_{13} term—that the LLM can reason about algebraically to guide targeted correction. This constitutes a qualitatively richer feedback mode than standard execution loops in the LLM literature [32, 33, 34, 35], since the oracle speaks in the same algebraic language as the problem domain rather than simply reporting execution success or failure.

Scope and limitations of this use case. This use case is prospective; no such discovery loop has yet been implemented. Several limitations must be noted. Detailed balance is *sufficient* but not necessary for a Boltzmann stationary distribution: algorithms that satisfy only global balance, such as ECMC [17], would correctly fail the checker and be excluded from consideration. Detailed balance says nothing about ergodicity (which must be checked separately) or about kinetic fidelity (whether the algorithm approximates physical dynamics). These are independent necessary conditions for a physically useful algorithm.

More subtly, the search space for new algorithms is vast and largely unstructured. Unconstrained LLM generation is likely to produce many syntactically valid but physically incoherent or non-ergodic algorithms. The most productive approach is constrained search: give the LLM a physically motivated algorithmic skeleton with a specific modifiable component, and ask it to fill in that component. Template-guided generation, combined with the formal checker as a hard correctness constraint, mirrors the approach of Fun-

Search [30] but with a formal mathematical oracle rather than an empirical evaluator—a qualitative upgrade in the nature of the correctness guarantee.

The physical kinetics target must also be operationalised. Detailed balance is a necessary condition for correct equilibrium sampling; it says nothing about whether the algorithm reproduces physically relevant dynamics (e.g., the correct relaxation timescale hierarchy or mean-squared displacement scaling). The physical fidelity metrics of Section 3.8 provide a continuous scalar signal for this, but a precise kinetic target must be specified before the loop can be used to optimise for it.

6.4 Physical kinetics characterisation

The physical fidelity metrics M1 and M2 (Section 3.8) enable quantitative comparison of the kinetic character of different MC algorithms targeting the same equilibrium distribution. This is useful in two settings.

First, when choosing among multiple correct (DB-passing) algorithms for a given model, M2 ranks their kinetic fidelity against Glauber reference dynamics and M1 compares their relaxation timescale structure. An algorithm with a more negative M2 accepts a kinetic departure from Brownian dynamics in exchange for faster decorrelation—a deliberate algorithmic acceleration.

Second, when the goal is to match the kinetics of a physical process as closely as possible (e.g., for time-resolved comparisons with experiment or with molecular dynamics), M2 and M1 provide a quantitative measure of how well the MC transition matrix approximates the physical one on small tractable systems. These metrics are computed numerically at specific parameter values; they do not constitute a formal proof of kinetic fidelity for all parameters.

7 Discussion

Scope and system size. The exponential cost of BFS— $\mathcal{O}(N^2 \cdot 2^d)$ in the number of states N and bit depth d —limits practical use to system sizes of roughly $\lesssim 10$ sites per dimension on current hardware. This is smaller than production simulation systems by several orders of magnitude. However, detailed-balance violations are a property of the algorithm, not the system size: an algorithm that fails DB on a 2×2 lattice fails it on a 200×200 lattice as well. Small systems are the correct setting for formal verification; large systems are the correct setting for production simulation. The workflow “certify on small, simulate on large” is analogous to formal verification of small kernel functions before deploying at scale.

What the checker does and does not prove. A PASS from the symbolic checker proves, under the assumptions listed in Section 3.9, that the Mathematica implementation of the algorithm satisfies detailed balance for all model parameter values, on the specific system sizes tested. It does not prove ergodicity (checked separately), kinetic fidelity

(assessed via the M1/M2 metrics), or that a translation of the algorithm to a production language preserves correctness. It also does not verify correctness for all possible system sizes, only those tested; in practice, the small system sizes that the checker can handle are sufficient to expose all structural detailed-balance violations, since these arise from the algorithm’s move logic rather than from finite-size effects.

Portability. Production MC simulations are written in C++, Fortran, Python, or Julia. The most defensible architecture for existing codebases is a two-level design: the production simulation runs in the native language; a reference Mathematica implementation of the MC move is maintained alongside and periodically tested against the production code on a comprehensive set of states and random seeds; and the symbolic checker is applied to the reference implementation. For new algorithm development the natural workflow is the reverse: design and certify in Mathematica, then translate once correctness is established.

The clean architecture of the `.wl` algorithm files—a small, bounded interface of four required definitions with no arbitrary I/O or polymorphic data structures—is also compatible with a minimal domain-specific language (DSL) approach: a single algorithm description with two interpreters, one producing the Mathematica checker input and one producing production C++ or Python. Because the DSL is restricted, the two interpreters can be verified semantically equivalent by inspection, and a PASS constitutes a proof valid for the production code as well. This is the strongest possible guarantee and the most natural design for new algorithm development.

Comparison with model checking. Formal model checkers such as PRISM and Storm could in principle verify temporal-logic properties of a Markov chain’s transition matrix. However, they require the user to construct the chain’s model by hand in a dedicated language—effectively reproducing the case analysis of an analytic proof. The present framework operates directly on the algorithm as executable code, with no model-translation step, and targets the detailed-balance condition directly as a pairwise algebraic identity. This combination—executable input, exact algebraic output, all parameter values simultaneously—is not provided by any existing formal method for Markov chains.

8 Conclusions

We have presented *DetailedBalanceChecker*, an automated framework for the exact symbolic verification of detailed balance in Monte Carlo algorithms. By intercepting all random calls and replacing them with reads from a binary tape, the framework constructs a complete symbolic transition matrix by BFS and checks the detailed-balance condition algebraically. Coupling constants, field functions, and inverse temperature are treated as free symbols throughout, so a PASS constitutes a mathematical proof valid for all parameter values simultaneously; a FAIL yields exact symbolic expressions identifying which transitions violate balance. Ergodicity is checked independently as a complement.

The rightward-only cluster algorithm provides a striking illustration of the method’s diagnostic value: it visits every state in the correct proportion under translation-invariant energies and passes all numerical statistical tests, yet violates detailed balance because the reverse move is never proposed. No histogram-based test can detect this; the symbolic checker catches it immediately. The example illustrates a class of bugs latent in algorithms for translation-invariant energies but manifest under any position-dependent field—precisely the case not tested in symmetric systems.

Symmetry-aware checking extends the method’s practical reach. By making the BFS produce syntactically identical expressions for G -orbit state pairs and verifying G -invariance post-hoc using generator sweeps, we achieve near $|G|$ -fold compression of the verification cost while maintaining correctness guarantees. On a 3×3 lattice with D4+translation symmetry, this compresses a two-particle component from 288 state pairs to 2 unique expressions, reducing the dominant FullSimplify cost from minutes to seconds.

The primary limitation is the exponential BFS cost, restricting practical verification to $\lesssim 10$ sites per dimension. This is sufficient to certify structural correctness of the algorithm—detailed-balance violations arise from algorithm logic, not from system size—but leaves open questions about large-system ergodicity and kinetic properties. Extensions under consideration include a DSL approach for direct verified translation to production code, and use of the checker as the formal oracle in an LLM-driven algorithm discovery loop, where the symbolic counterexample can guide targeted revisions in a way that runtime error feedback cannot.

A BitsToState: bijective state enumeration

The reference implementation uses a bijective map between non-negative integers and labeled-particle arrays. An integer ID encodes, in order: (i) the array length L , via a prefix count $\sum_{l < L} |\mathcal{S}_l|$ where $|\mathcal{S}_l|$ is the total number of labeled arrays of length l ; (ii) the particle number N , via a secondary prefix; (iii) the sorted positions of the N particles, ranked by the combinatorial number system; (iv) the assignment of types to positions, ranked by the factorial number system. Each step has an explicit inverse.

The function `BITSTOSTATE` interprets a bit string as the binary representation $ID = \sum_i b_i \cdot 2^{|b|-i}$ and calls the decoder. States are enumerated in strictly increasing order of complexity; algorithm files may supply a `ValidStateIDs` filter that returns null for configurations not satisfying global constraints (e.g., perfect-square lattice size for 2D systems), causing the checker to skip those strings without any BFS work.

B Supported random call types

Table 6: Random call types supported by the checker and their reduction to the binary tape.

Call	Reduction
<code>RandomReal[]</code>	Interval-tracking token
<code>RandomInteger[{lo,hi}]</code>	Rejection sampling, $\lceil \log_2(h-l+1) \rceil$ bits
<code>RandomChoice[list]</code>	Rejection sampling over elements
<code>RandomChoice[w->els]</code>	Sequential Bernoulli decomposition
<code>RandomVariate[Normal[μ,σ]]</code>	Truncated Gaussian, Eq. (3)
<code>RandomPermutation[n]</code>	Knuth shuffle via <code>RandomInteger</code>
<code>RandomSample[list,k]</code>	Partial Knuth shuffle

References

- [1] N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller, and E. Teller, *Equation of State Calculations by Fast Computing Machines*, J. Chem. Phys. **21**, 1087 (1953).
- [2] W. K. Hastings, *Monte Carlo Sampling Methods Using Markov Chains and Their Applications*, Biometrika **57**, 97 (1970).
- [3] D. Frenkel and B. Smit, *Understanding Molecular Simulation: From Algorithms to Applications*, 2nd ed., Academic Press (2002).
- [4] M. P. Allen and D. J. Tildesley, *Computer Simulation of Liquids*, Oxford University Press (1987).
- [5] M. E. J. Newman and G. T. Barkema, *Monte Carlo Methods in Statistical Physics*, Oxford University Press (1999).
- [6] D. A. Levin, Y. Peres, and E. L. Wilmer, *Markov Chains and Mixing Times*, American Mathematical Society (2009).
- [7] K. Kawasaki, *Diffusion Constants near the Critical Point for Time-Dependent Ising Models I*, Phys. Rev. **145**, 224 (1966).
- [8] S. Whitelam and P. L. Geissler, *Avoiding Unphysical Kinetic Traps in Monte Carlo Simulations of Strongly Attractive Particles*, J. Chem. Phys. **127**, 154101 (2007).
- [9] S. Whitelam, E. H. Feng, M. F. Hagan, and P. L. Geissler, *The Role of Collective Motion in Examples of Coarsening and Self-Assembly*, Soft Matter **5**, 1251 (2009).
- [10] R. H. Swendsen and J.-S. Wang, *Nonuniversal Critical Dynamics in Monte Carlo Simulations*, Phys. Rev. Lett. **58**, 86 (1987).
- [11] U. Wolff, *Collective Monte Carlo Updating for Spin Systems*, Phys. Rev. Lett. **62**, 361 (1989).

- [12] J. Liu and E. Luijten, *Rejection-Free Geometric Cluster Algorithm for Complex Fluids*, Phys. Rev. Lett. **92**, 035504 (2004).
- [13] J. Geweke, *Getting it Right: Joint Distribution Tests of Posterior Simulators*, J. Am. Stat. Assoc. **99**, 799 (2004).
- [14] F. P. Kelly, *Reversibility and Stochastic Networks*, Wiley (1979).
- [15] J. R. Norris, *Markov Chains*, Cambridge University Press (1998).
- [16] V. I. Manousiouthakis and M. W. Deem, *Strict Detailed Balance is Unnecessary in Monte Carlo Simulation*, Phys. Rev. E **59**, 2 (1999).
- [17] E. P. Bernard, W. Krauth, and D. B. Wilson, *Event-Chain Monte Carlo Algorithms for Hard-Sphere Systems*, Phys. Rev. E **80**, 056704 (2009).
- [18] R. J. Glauber, *Time-Dependent Statistics of the Ising Model*, J. Math. Phys. **4**, 294 (1963).
- [19] A. M. Turing, *On Computable Numbers, with an Application to the Entscheidungsproblem*, Proc. London Math. Soc. **s2-42**, 230 (1936).
- [20] R. A. Horn and C. R. Johnson, *Matrix Analysis*, Cambridge University Press (1985).
- [21] C. Daws, *Symbolic and Parametric Model Checking of Discrete-Time Markov Chains*, in *Proc. ICTAC 2004*, LNCS 3407, Springer (2005).
- [22] M. Kwiatkowska, G. Norman, and D. Parker, *PRISM 4.0: Verification of Probabilistic Real-Time Systems*, in *Proc. CAV 2011*, LNCS 6806, pp. 585–591, Springer (2011).
- [23] C. Dehnert, S. Junges, J.-P. Katoen, and M. Volk, *A Storm is Coming: A Modern Probabilistic Model Checker*, in *Proc. CAV 2017*, LNCS 10427, pp. 592–600, Springer (2017).
- [24] T. Gehr, S. Misailovic, and M. Vechev, *PSI: Exact Symbolic Inference for Probabilistic Programs*, in *Proc. PLDI 2016*, pp. 662–677, ACM (2016).
- [25] A. Meurer *et al.*, *SymPy: Symbolic Mathematics in Python*, PeerJ Computer Science **3**, e103 (2017).
- [26] J. Bezanson, A. Edelman, S. Karpinski, and V. B. Shah, *Julia: A Fresh Approach to Numerical Computing*, SIAM Review **59**(1), 65–98 (2017).
- [27] S. Gowda, Y. Ma, A. Cheli, M. Gwózdź, V. B. Shah, A. Edelman, and C. Rackauckas, *Symbolics.jl: A Modern Computer Algebra System for a Modern Language*, arXiv:2105.03949 (2021).
- [28] J. Revels, *Cassette.jl: Overdubbing Context-Dependent Execution in Julia*, <https://github.com/JuliaLabs/Cassette.jl> (2018).
- [29] Wolfram Research, *Mathematica, Version 14*, Wolfram Research, Inc., Champaign, IL (2024).
- [30] B. Romera-Paredes *et al.*, *Mathematical Discoveries from Program Search with Large Language Models*, Nature **628**, 73 (2024).
- [31] Google DeepMind, *AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery*, arXiv:2506.13131 (2025).
- [32] L. Gao *et al.*, *PAL: Program-Aided Language Models*, Proc. ICML, PMLR 202:10764–10799 (2023).
- [33] S. Yao *et al.*, *ReAct: Synergizing Reasoning and Acting in Language Models*, ICLR (2023).
- [34] N. Shinn *et al.*, *Reflexion: Language Agents with Verbal Reinforcement Learning*, NeurIPS (2023).
- [35] *LLMLOOP: Automating Refinement of LLM-Generated Code through Iterative Feedback*, ICSME, arXiv:2603.23613 (2025).
- [36] J. I. Siepmann and D. Frenkel, *Configurational Bias Monte Carlo: A New Sampling Scheme for Flexible Chains*, Mol. Phys. **75**, 59 (1992).
- [37] L. O. Hedges, *libVMMC: Rejection-Free Monte Carlo for Hard Spheres*, <https://github.com/lohedges/vmmc> (2015).
- [38] Wolfram Research, *WolframClientForPython*, <https://github.com/WolframResearch/WolframClientForPython> (2019).